



UNIVERSIDADE FEDERAL DE SERGIPE
CENTRO DE CIÊNCIAS EXATAS E TECNOLOGIA
DEPARTAMENTO DE COMPUTAÇÃO

CAIO VINICIUS SOUZA SANTOS
LUCAS THIERY OLIVEIRA DOS SANTOS
PALOMA DE SANTANA ARAUJO MOTA
PAULO VINICIUS DE JESUS CUNHA

RELATÓRIO DO JOGO: LIGUE 4++

SÃO CRISTÓVÃO – SE
2026

SUMÁRIO

1. Introdução
2. Fundamentação Teórica
3. Metodologia de Desenvolvimento
4. Estrutura do Jogo e Arquitetura
5. Desenvolvimento e Implementação (Trechos de Código)
6. Resultados e Discussões
7. Conclusões



1. Introdução

O projeto consiste na implementação do jogo Ligue 4++, uma versão aprimorada do clássico jogo de conexão de quatro peças em uma grade vertical 6×7 . A versão inclui fichas especiais (explosivas e portais) e múltiplos modos de jogo: Jogador vs Jogador, Jogador vs Computador e Automático.

Objetivos:

- Demonstrar a aplicação prática dos conceitos de programação imperativa em C.
- Implementar um jogo interativo com regras complexas, validações e múltiplos modos.
- Exercitar o uso de estruturas de dados, controle de fluxo, funções e recursão (para explosões em cadeia, a ser implementado).
- Linguagem: C
- Bibliotecas padrão utilizadas: `stdio.h`, `stdlib.h`, `time.h`, `string.h`, `locale.h`.
- Ambiente: ide dev C++

2. Fundamentação Teórica

O projeto utiliza os seguintes conceitos da linguagem C:

- Matrizes bidimensionais para representação do tabuleiro 6×7
- Estruturas (struct) para armazenamento de dados dos campeões
- Recursão para implementação de explosões em cadeia
- Controle de fluxo com estruturas condicionais e loops
- Funções da biblioteca padrão para entrada/saída e aleatoriedade

3. Metodologia de Desenvolvimento

O código foi desenvolvido seguindo abordagem incremental:

1. Implementação do tabuleiro básico e inserção de fichas comuns
2. Adição das fichas especiais (explosivas e portais)
3. Implementação dos três modos de jogo
4. Desenvolvimento do sistema de ranking com arquivo
5. Refinamento da interface com cores e tratamento de erros

4. Estrutura do Jogo e Arquitetura

4.1 Representação do Tabuleiro

```
char tab[6][7]; // Matriz 6 linhas × 7 colunas
```

- ' ' = posição vazia

- 'X' = ficha comum jogador 1 (amarelo)

- 'O' = ficha comum jogador 2 (azul)

- 'A' = ficha explosiva ativa jogador X (definido por `EXPL\_ATIVA\_X`)

- 'B' = ficha explosiva ativa jogador O (definido por `EXPL\_ATIVA\_O`)

- 'X' = ficha explosiva desativada jogador X (definido por `EXPL\_DESAT\_X`)

- 'O' = ficha explosiva desativada jogador O (definido por `EXPL\_DESAT\_O`)

- '@' = ficha portal (definido por `portal`)

4.2 Variáveis de Controle

char jogador = 'X';	→ Jogador atual
int ganhador = 0;	→ 0=em jogo, 1=vitória, 2=empate
int rodada = 0;	→ Contador de rodadas
int explosivasX = 0;	→ Fichas explosivas jogador X
int portalX = 0;	→ Fichas portal jogador X
int explosivasO = 0;	→ Fichas explosivas jogador O
int portalO = 0;	→ Fichas portal jogador O
char nomeJogador1[50];	→ Nome do jogador 1
char nomeJogador2[50];	→ Nome do jogador 2/Computador

5. Desenvolvimento e Implementação (Trechos de Código)

- Como foi feita a leitura do arquivo?

Nós fizemos a leitura do arquivo usando a função `fscanf` dentro de um laço `while`. A cada repetição, o programa tenta ler o nome do jogador e o número de jogadas salvas no arquivo. Enquanto não chega ao final do arquivo (EOF), nós vamos guardando esses dados no vetor `campeoes` e incrementando a variável `total` para contar quantos registros foram carregados. Dessa forma, nós conseguimos importar todos os dados que já estavam salvos no ranking e usar essas informações depois, como para mostrar os melhores jogadores.

```
// leitura do arquivo
while(fscanf(arquivo,"%s %d", campeoes[total].nome, &campeoes[total].jogadas) != EOF){
    total++;
}

fclose(arquivo);
```

- Qual estrutura representa as fichas e o tabuleiro?

Nós representamos o tabuleiro usando uma matriz de caracteres de 6 linhas por 7 colunas (char tab[6][7]). Cada posição dessa matriz guarda qual peça está ali ou um espaço vazio, o que facilita tanto a visualização quanto a verificação do jogo.

As fichas especiais, como as explosivas e os portais, nós controlamos usando variáveis inteiras separadas para cada jogador, como explosivasX, portalX, explosivasO e portalO.

Além disso, nós usamos constantes em forma de caracteres para identificar cada tipo de peça no jogo, como peças ativas, desativadas e portais. Também criamos variáveis para controlar o jogador da vez, a rodada atual e o estado da partida, indicando se alguém ganhou ou se houve empate.

```
#define EXPL_ATIVA_X 'A'
#define EXPL_ATIVA_O 'B'
#define EXPL_DESAT_X 'X'
#define EXPL_DESAT_O 'O'
#define portal '@'

// Variavel que controla o jogador atual
char jogador = 'X';

// Matriz que representa o tabuleiro (6 Linhas x 7 colunas)
char tab[6][7];
char nomeJogador1[50];
char nomeJogador2[50];

// Variaveis de controle do jogo
int ganhador = 0, empatou = 1;
int linha, coluna;
int rodada = 0;
int i,j;

// fichas do jogador x
int explosivasX = 0;
int portalX = 0;

// fichas do jogador o
int explosivasO = 0;
int portalO = 0;
```

- Como o programa descobre que o jogo acabou?

Nós fizemos isso usando a função vencer(). Nessa função, nós verificamos o tabuleiro inteiro procurando uma sequência de quatro peças iguais. Para isso, nós checamos quatro possibilidades:

- 1-Na horizontal
- 2-Na vertical
- 3-Na diagonal principal
- 4-Na diagonal secundária

Se em qualquer uma dessas verificações nós encontramos quatro peças iguais em sequência, o programa define que existe um vencedor e o jogo é encerrado. Além disso, o nosso jogo será declarado empate quando ambos os jogadores não tiverem mais fichas comuns. Se isso acontecer e ninguém tiver vencido, nós consideramos que o jogo terminou em empate. Se nenhuma dessas situações acontecer, o jogo continua normalmente.

```
//Função vencer(verificar se há um vencedor ou empate)
int vencer() {
    int i, j;

    // Verifica o horizontal
    for (i = 0; i < 6; i++) {
        for (j = 0; j < 4; j++) {
            if (
                tab[i][j] != ' ' &&
                tab[i][j] == tab[i][j+1] &&
                tab[i][j] == tab[i][j+2] &&
                tab[i][j] == tab[i][j+3]
            ) {
                ganhador = 1;
                return 1;
            }
        }
    }

    // Verifica o vertical
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 7; j++) {
            if (tab[i][j] != ' ' &&
                tab[i][j] == tab[i+1][j] &&
                tab[i][j] == tab[i+2][j] &&
                tab[i][j] == tab[i+3][j]) {
                ganhador = 1;
                return 1;
            }
        }
    }

    // Verifica o da diagonal principal
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 4; j++) {
            if (tab[i][j] != ' ' &&
                tab[i][j] == tab[i+1][j+1] &&
                tab[i][j] == tab[i+2][j+2] &&
                tab[i][j] == tab[i+3][j+3]) {
                ganhador = 1;
                return 1;
            }
        }
    }

    // Verifica o da diagonal secundária
    for (i = 3; i < 6; i++) {
        for (j = 0; j < 4; j++) {
            if (tab[i][j] != ' ' &&
                tab[i][j] == tab[i-1][j+1] &&
                tab[i][j] == tab[i-2][j+2] &&
                tab[i][j] == tab[i-3][j+3]) {
                ganhador = 1;
                return 1;
            }
        }
    }

    // Verificação de empate
    if (comunsX == 0 && comunsO == 0) {
        ganhador = 2;
        return 2;
    }
}
```

- Como o programa descobre quem venceu?

Quando a variável ganhador indica que alguém venceu, nós verificamos qual jogador fez a última jogada.

Se o jogador atual for 'X', nós consideramos que o vencedor é o jogador 1. Caso contrário, o vencedor é o jogador 2.

Depois disso, nós mostramos na tela uma mensagem de parabéns com o nome do jogador vencedor e chamamos a função RegistroDoHall, que salva o nome do vencedor e a quantidade de rodadas no ranking do jogo.

```
if(ganhador == 1){
    if(jogador == 'X'){
        printf(YELLOW "PARABENS %s! VOC VENCEU!" RESET, nomeJogador1);
        RegistroDoHall(nomeJogador1,rodada);
    }
    else {
        printf(BLUE "PARABENS %s! VOC VENCEU!" RESET, nomeJogador2);
        RegistroDoHall(nomeJogador2,rodada);
    }
}
else if(ganhador == 2) {
    printf(GREEN "O JOGO EMPATOU!" RESET);
}
```

- Como as rodadas foram implementadas?

Nós controlamos as rodadas usando uma variável inteira chamada rodada. Toda vez que uma jogada acontece, seja de um jogador humano ou da máquina, nós incrementamos essa variável em 1. Além disso, nós criamos a função AtualizarFichas(), que usa o número da rodada para liberar fichas especiais. A cada 5 rodadas, nós adicionamos uma ficha explosiva e uma ficha portal para cada jogador. Assim, as rodadas servem tanto para controlar o andamento da partida quanto para liberar recursos extras no jogo.

```
// vai atualizar o numero de fichas especiais de cada jogador e maquina
void AtualizarFichas(){
    // a cada cinco rodadas dar aos jogadores as fichas portais e explosivas
    if(rodada > 0 && rodada % 5 == 0){
        portalX++;
        portalO++;
        explosivasX++;
        explosivasO++;
    }
}
```

- Como o programa escaneia o nome dos jogadores?

Nós fizemos isso usando a função DefinirNomes(int modo). Dependendo do modo de jogo escolhido, o programa pede os nomes de formas diferentes.

-No modo jogador contra jogador, nós pedimos o nome dos dois jogadores usando fgets.

-No modo jogador contra máquina, nós pedimos o nome de apenas um jogador e definimos o outro automaticamente como “Computador”.

-No modo automático, nós definimos os nomes como “Alpha” e “Beta”.

Depois de ler os nomes, nós removemos o caractere de quebra de linha (\n) para evitar problemas na hora de mostrar os nomes na tela.

```
//Responsavel por escanear o nome dos jogadores
void DefinirNomes(int modo){

    if(modo == 1){
        printf(YELLOW "Digite o nome do jogador 1: " RESET);
        fgets(nomeJogador1, sizeof(nomeJogador1), stdin);
        nomeJogador1[strcspn(nomeJogador1, "\n")] = '\0';

        printf(BLUE "Digite o nome do jogador 2: " RESET);
        fgets(nomeJogador2, sizeof(nomeJogador2), stdin);
        nomeJogador2[strcspn(nomeJogador2, "\n")] = '\0';
    }

    else if (modo == 2) {
        printf(YELLOW "Digite o nome do Jogador: " RESET);
        fgets(nomeJogador1, sizeof(nomeJogador1), stdin);
        nomeJogador1[strcspn(nomeJogador1, "\n")] = '\0';
        strcpy(nomeJogador2, "Computador");
    }

    else if (modo == 3) {
        strcpy(nomeJogador1, "Alpha");
        strcpy(nomeJogador2, "Beta");
    }
}
```

- Como exibir os três melhores?

Nós exibimos os três melhores jogadores usando os dados que foram lidos do arquivo e guardados no vetor de campeões.

Depois de organizar esses dados, nós mostramos na tela os três primeiros colocados do ranking, exibindo o nome e a quantidade de jogadas de cada um.

Assim, o jogador consegue ver quais foram os melhores resultados já registrados no jogo.

```
// vai exibir somente os tops 3
int limite;

if(total < 3){
    limite = total;
}

else{
    limite = 3;
}

printf("\n" CYAN " +-----+ " RESET "\n");
printf(CYAN " " RESET BOLD " HALL DOS CAMPE ES " RESET CYAN " " RESET "\n");
printf(CYAN " ----- " RESET "\n");
printf(" \n");

for(i = 0; i< limite; i++){

    if(i == 0){
        printf(" " GOLD "1 " RESET " %-15s " GOLD " " RESET " %2d jogadas \n", campeoes[i].nome, campeoes[i].jogadas);
    }

    if(i == 1){
        printf(" " SILVER "2 " RESET " %-15s " SILVER " " RESET " %2d jogadas \n", campeoes[i].nome, campeoes[i].jogadas);
    }

    if(i == 2){
        printf(" " BRONZE "3 " RESET " %-15s " BRONZE " " RESET " %2d jogadas \n", campeoes[i].nome, campeoes[i].jogadas);
    }
    printf(" \n");
    printf(CYAN " +-----+ " RESET "\n");
}
}
```

- Como é a estrutura do ranking?

Nós organizamos o ranking usando uma estrutura (struct) que guarda, pelo menos, o nome do jogador e a quantidade de jogadas.

Essas estruturas ficam armazenadas em um vetor, que é preenchido quando nós lemos o arquivo no início do programa.

Quando uma partida termina, nós adicionamos o vencedor a esse conjunto de dados e atualizamos o arquivo. Dessa forma, o ranking funciona como um histórico dos resultados do jogo.

```
// criacao dos dados do HALL
typedef struct {
    char nome[50];
    int jogadas;
}Campeao;
```

- Como funciona o menu para escolher o modo de jogo?

Nós criamos a função ModoJogo() para permitir que o jogador escolha como a partida vai acontecer. Primeiro, nós limpamos a tela para organizar a interface e depois mostramos as três opções de modo de jogo: jogador contra jogador, jogador contra máquina e modo automático. Em seguida, nós pedimos para o usuário digitar a opção desejada e fazemos a leitura com scanf. Nós também usamos uma variável para verificar se a leitura deu certo.

Se o jogador digitar letras ou um número inválido, nós entramos em um laço de repetição que mostra uma mensagem de erro e pede a opção novamente. Esse processo só termina quando o usuário digita um valor válido entre 1 e 3. Assim, nós garantimos que o programa sempre continue com um modo de jogo correto escolhido.

```
//Menu para escolher modo de jogo
int ModoJogo(){

    LimparTela();
    int modo;

    printf("\n" CYAN " +-----+ RESET "\n");
    printf(CYAN " " RESET BOLD " MODO DE JOGO " RESET CYAN " " RESET "\n");
    printf(CYAN " ----- " RESET "\n");
    printf(" \n");

    // primeiro Lugar
    printf(" " GREEN BOLD "[1]" RESET BOLD " Jogador vs Jogador" RESET " \n");
    printf(" " GREEN "---- Cl ssico local ----" RESET " \n");
    printf(" \n");

    // segundo Lugar
    printf(" " YELLOW BOLD "[2]" RESET BOLD " Jogador vs M quina" RESET " \n");
    printf(" " YELLOW "---- Desafie a CPU ----" RESET " \n");
    printf(" \n");

    // terceiro Lugar
    printf(" " MAGENTA BOLD "[3]" RESET BOLD " M quina vs M quina" RESET " \n");
    printf(" " MAGENTA "---- Modo Autom tico ----" RESET " \n");
```

```

printf("                                \n");

printf(CYAN " +-----+ RESET \n");
printf(BOLD "\n Escolha o modo de jogo: " RESET);

int leu = scanf("%d", &modo);

while(leu == 0 || modo < 1 || modo > 3) {

    if(leu == 0) {
        //joga fora as letras digitadas erradas
        fflush(stdin);
        printf("Entrada invalida! Digite apenas NUMEROS: ");
    } else {
        printf("Opcao invalida! Digite 1, 2 ou 3: ");
    }

    // Tenta ler novamente
    leu = scanf("%d", &modo);
}

```

6. Resultados e Discussões

6.1 Funcionalidades Implementadas

- [x] Tabuleiro 6×7
- [x] Três modos de jogo: Jogador×Jogador, Jogador×Computador, Automático
- [x] Fichas especiais: explosivas (com recursão) e portais
- [x] Sistema de ranking com arquivo persistente
- [x] Interface com menus e tratamento de erros

6.2 Tratamento de Erros

- Validação de entrada numérica (scanf com limpeza de buffer)
- Verificação de coluna cheia
- Verificação de fichas disponíveis

6.3 Estruturas de Dados Utilizadas

1. Matriz (char[6][7]) para o tabuleiro
2. Array de struct (Campeao[100]) para ranking
3. Variáveis de controle para estado do jogo
4. Strings (char[]) para nomes dos jogadores

6.4 Recursos da Linguagem Utilizados

- Funções recursivas para explosão em cadeia
- Manipulação de arquivos com (fopen, fscanf, fprintf)
- Funções de tempo para delay (Sleep/sleep)
- Aleatoriedade com (rand() e srand())

7. Conclusões

O jogo Ligue 4++ foi implementado conforme especificado, com todas as funcionalidades requeridas: tabuleiro funcional, fichas especiais (explosivas e portais), três modos de jogo e sistema de ranking persistente. O código demonstra aplicação prática de conceitos fundamentais de programação em C, incluindo estruturas de dados, recursão, manipulação de arquivos e controle de fluxo. A implementação segue boas práticas de programação com código modular, tratamento de erros e interface amigável.